

MP 3: Cryptography

This project is split into two parts, with the first checkpoint due on **October 30, 2025 at 6:00pm** and the second checkpoint due on **November 6, 2025 at 6:00pm**. The first checkpoint is worth 20 points, and the second checkpoint is worth 80 points. We strongly recommend that you get started early.

The code and other answers your group submits **MUST** be entirely your own work, and you are bound by the Student Code. You **MAY** consult with other students about the conceptualization of the project and the meaning of the questions, but you **MUST NOT** look at any part of someone else's solution. You may consult published references, provided that you appropriately cite them (e.g., with program comments), as you would in an academic paper.

Solutions **MUST** be submitted electronically on the master branch of your class Git repo.

"Anyone, from the most clueless amateur to the best cryptographer, can create an algorithm that he himself can't break."

– Bruce Schneier

Introduction

In this project, you will be using cryptographic libraries to decrypt multiple types of ciphers, break them, and launch attacks on widely used cryptographic hash functions. In 3.1.2, you will be decrypting ciphers with given ciphertexts and key values. Then, you will use similar techniques to break a weak cipher with a limited key space. In 3.1.6, you will build a weak hash algorithm and find a collision for a given string. In 3.2.1, we will guide you through attacking the authentication functionality of an imaginary server API. The attack will exploit the length-extension vulnerability of hash functions in the MD5 and SHA1&2 families. In 3.2.2, you will use a cryptanalyst tool to generate different messages with the same MD5 hash value (collisions). You'll then investigate how that capability can be exploited to conceal malicious behavior in software. In 3.2.3, you will be performing a padding oracle attack using AES-encrypted ciphertext. Last, in 3.2.4, you will implement another attack on RSA, this time to implement the "Mining Your Ps and Qs" attack.

Objectives:

- Become familiar with existing cryptographic libraries and how to utilize them
- Understand pitfalls in cryptography and appreciate why you *should not write your own cryptographic libraries*
- Execute a classic cryptographic attack on MD5 and other broken cryptographic algorithms
- Execute a length-extension attack similar to a historical attack executed successfully against Flickr.

Guidelines

- You MUST use a Python version ≥ 3.5 and < 3.10 .
- Your answers may or may not be the same as your classmates'.
- All the necessary files to start the project will be given under the folder called "Crypto" in your git repository. We've also generated some empty files in which you will submit your answers. You MUST submit your answers in the provided files; we will only grade what's there!

3.1 Checkpoint 1 (20 points)

Python3 tutorial

In this section, you will be writing several python3 scripts to do string encoding and manipulations needed to correctly read our input files and submit your answers.

Reading .hex files

In the later parts of this MP, you will be reading in .hex files, which are plaintext files containing an ASCII string representation of a single hexadecimal number. This is the content of an example .hex file:

```
3dab821d92b5ca7f48beee066996b8abc82f7e5646a0561710ea5bc11c80
```

The following Python3 code snippet will read the contents of the file as a string and store it in `file_content`:

```
# strip() remove any leading or trailing whitespace characters
with open('file_name') as f:
    file_content = f.read().strip()
```

From here, there's a number of things that you could do. Depending on the cryptographic library that you are using, you may need to use different data types, but here we list the most common conversions that you may need:

```
# parse the string into a bytes object representing the hexadecimal number
binary_content = bytes.fromhex(file_content)

# parse the string into integer
integer_parsed = int(file_content,16)

# parse an integer to a hex string and remove the leading '0x'
str = hex(integer_parsed)[2:]

# parse an integer to a binary string and remove the leading '0b'
str = bin(integer_parsed)[2:]
```

3.1.1 Exercise (2 points)

(Difficulty: Easy)

Files

1. `3.1.1_value.hex`: an ASCII string representing a hexadecimal value

Based on what you learned in the previous section, we want to convert the given value into different representations and submit them in the specified files.

What to submit

1. Convert the value in `3.1.1_value.hex` to decimal and submit the decimal number as a string in `sol_3.1.1_decimal.txt`
2. Convert the value in `3.1.1_value.hex` to binary and submit the binary number as a string in `sol_3.1.1_binary.txt`

Symmetric Encryption, Public Key Encryption, and Cryptographic Hashes

In this section, you will be writing your own cryptographic library to decrypt a substitution cipher, and using existing cryptographic libraries to experiment with a symmetric encryption called AES and a public key encryption called RSA.

3.1.2 Vigenere Cipher (3 points)

(Difficulty: Easy)

Files

1. `3.1.2_vigenere_ciphertext.txt`: ciphertext

`3.1.2_vigenere_ciphertext.txt` contains a ciphertext for a Vigenere cipher. The plaintext we encrypted is a concatenation of ten clues from the game show Jeopardy and has only uppercase letters, numbers, and spaces. Numbers and spaces in the plaintext were not encrypted; they appear exactly as they did in the plaintext.

Your task is to write a Python3 script in `sol_3.1.2.py` that decrypts the given Vigenere ciphertext and writes the plaintext to the file `sol_3.1.2.txt`. Your script must take three arguments from the command line: the ciphertext file, a file containing the reference text used in your frequency analysis, and the output file. You should use a reference text to calculate the letter frequency for your attack; include it in your submission as `sol_3.1.2_reference.txt` and it will be passed as an argument to your script. We will run your script as follows:

```
$ python3 your_script.py ciphertext_file reference_text_file output_file
```

What to submit Your Python3 script in `sol_3.1.2.py`, your reference text file in `sol_3.1.2_reference.txt`, and your plaintext in `sol_3.1.2.txt`.

3.1.3 Decrypting a ciphertext with AES (3 points)

(Difficulty: Easy)

Files

1. 3.1.3_aes_key.hex: key
2. 3.1.3_aes_iv.hex: initialization vector
3. 3.1.3_aes_ciphertext.hex: ciphertext

3.1.3_aes_key.hex contains a 256-bit AES key represented as an ASCII string of hexadecimal values. 3.1.3_aes_iv.hex contains a 128-bit initialization vector in a similar representation. We encrypted a Jeopardy clue using AES in CBC mode using this key and initialization vector and wrote the resulting ciphertext (also stored in hexadecimal) to 3.1.3_aes_ciphertext.hex. Create a Python3 script named sol_3.1.3.py that decrypts the ciphertext using the provided information and outputs the plaintext to a specified file. Your script must take four arguments from the command line: the ciphertext file, the key file, the initialization vector file, and the output file. We will run your script as follows:

```
$ python3 your_script.py ciphertext_file key_file iv_file output_file
```

Cryptographic Library

For this MP, we allow PyCryptodome, an open-source crypto library. PyCryptodome can be installed using pip3 with `sudo pip3 install pycryptodome` or by going to their website at <https://www.pycryptodome.org/>.

What to submit Your Python3 script in sol_3.1.3.py and the decrypted message in sol_3.1.3.txt.

3.1.4 AES: Breaking A Weak AES Key (3 points)

(Difficulty: Easy)

Files

1. 3.1.4_aes_weak_ciphertext.hex: ciphertext

As with the last task, we encrypted a Jeopardy clue using 256-bit AES in CBC mode and stored the result in hexadecimal in the file 3.1.4_aes_weak_ciphertext.hex. But for this task, we haven't supplied the key. All we will tell you about the key is that it is 256 bits long and its 251 most significant (leftmost) bits are all 0s. The initialization vector was set to all 0s. First, find all plaintexts in the given key space. Then, you will review the plaintexts to find the correct plaintext that is the Jeopardy clue and the corresponding key.

What to submit Find the **key** of the appropriate plaintext and submit it as a hex string in sol_3.1.4.hex. Remember that this AES key is 256 bits long.

3.1.5 Decrypting a ciphertext with RSA (3 points)

(Difficulty: Easy)

Files

1. 3.1.5_RSA_private_key.hex: RSA private key (d) as hexadecimal string
2. 3.1.5_RSA_modulo.hex: RSA modulo (N) as hexadecimal string
3. 3.1.5_RSA_ciphertext.hex: an encrypted prime number that is encrypted with 1024-bit RSA as a hexadecimal string

In this part, we used 1024-bit textbook RSA to encrypt a prime number using your public key and stored it in 3.1.5_RSA_ciphertext.hex as a hex string. Create a Python3 script named sol_3.1.5.py that takes as arguments the ciphertext, the private key, and the RSA modulo to compute the plaintext prime number and write it as a hex string to a specified file. We will run your script as follows:

```
$ python3 your_script.py ciphertext_file key_file modulo_file output_file
```

What to submit Your Python3 script in sol_3.1.5.py and the prime number as a hex string in sol_3.1.5.hex.

Hash Functions

This section will give you a chance to explore cryptographic hashing using existing cryptographic libraries and illustrate the potential pitfalls of writing your own cryptographic functions.

3.1.6 Weak Hashing Algorithm (6 points)

(Difficulty: Medium)

Files

1. 3.1.6_input_string.txt: input string

Below you'll find the pseudocode for a weak hashing algorithm we're calling WHA. It operates on bytes (block size of 8 bits) and outputs a 32-bit hash.

```
WHA:
Input{inStr: a binary string of bytes}
Output{outHash: 32-bit hashcode for the inStr as a series of hex values}
Mask: 0x3FFFFFFF
outHash: 0
for byte in input
    intermediate_value = ((byte XOR 0xCC) Left Shift 24) OR
                        ((byte XOR 0x33) Left Shift 16) OR
                        ((byte XOR 0xAA) Left Shift 8) OR
                        (byte XOR 0x55)
```

```
    outHash = (outHash AND Mask) + (intermediate_value AND Mask)
return outHash
```

First, you'll need to implement WHA in Python. Here are some sample inputs you can use to test your implementation: `WHA("Hello world!") = 0x50b027cf` and `WHA("I am Groot.") = 0x57293cbb`

In the file `3.1.6_input_string.txt`, you'll find another Jeopardy clue (surprise!). Your goal is to find another string that produces the same WHA output as this Jeopardy clue. In other words, demonstrate that this hash is not second preimage resistant.

Find a string with the same WHA output as `3.1.6_input_string.txt` and submit it in `sol_3.1.6.txt`. Also, submit the code for your implementation of the WHA algorithm in `sol_3.1.6.py`. Your Python3 script should take as arguments a text file and an output file, and outputs the WHA hash of the content of the file as a hex string in the specified file. We will run your script as follows:

```
$ python3 your_script.py file.txt output_file
```

What to submit Your Python3 script in `sol_3.1.6.py` and the collision string in `sol_3.1.6.txt`

Checkpoint 1: Submission Checklist

The following blank files for checkpoint 1 have been created in your git repository under the directory Crypto. Put your solutions inside the corresponding files, then commit it to git. All .hex and .txt files MUST be submitted as ASCII plaintext, and any lines in .txt and .hex submissions that begin with a '#' will be ignored.

- sol_3.1.1_decimal.txt
- sol_3.1.1_binary.txt
- sol_3.1.2.py
- sol_3.1.2_reference.txt
- sol_3.1.2.txt
- sol_3.1.3.py
- sol_3.1.3.txt
- sol_3.1.4.hex
- sol_3.1.5.py
- sol_3.1.5.hex
- sol_3.1.6.py
- sol_3.1.6.txt

Example content of a .txt solution file

```
# this line is ignored
SPN WMKTQIW QR SPBW HQGRSEMW HQVS QY VEKW
```

Example content of a .hex solution file

```
# this line is also ignored
3dab821d92b5ca7f48beee066996b8abc82f7e5646a0561710ea5bc11c80d
```


3.2 Checkpoint 2 (80 points)

3.2.1 Length Extension (15 points)

(Difficulty: Medium)

We have to be careful with the way we construct our Hash-based Message Authentication Codes (HMACs). HMACs use a symmetric key to guarantee a message's integrity as well as ensure that only someone with the key could have generated the HMAC. Some HMAC constructions are subject to *length extension* attacks when using particular hash functions.

Most of the hash functions we've discussed (MD5, SHA1, SHA256) use a design called the Merkle-Damgård construction. Each is built around a *compression function* f and maintains an internal state s , which is initialized to a fixed constant. Messages are processed in fixed-size blocks by applying the compression function to the current state and the current block to compute an updated internal state, i.e. $s_{i+1} = f(s_i, b_i)$. The result of the final application of the compression function becomes the output of the hash function. Thus, the output of these hash functions also leaks the internal state of the algorithm!

A consequence of this design is that if we know the hash of an n -block message, we can find the hash of longer messages by applying the compression function for each block $b_{n+1}, b_{n+2}, \dots$ that we want to add. This process is called length extension.

Experiment with Length Extension in Python

To experiment with this idea, we'll use a Python3 implementation of the MD5 hash function, though HMACs built with SHA-1 and SHA-256 can be vulnerable to length extension in the same way. You should have a `pymd5.py` module in your Git repository. Documentation for `pymd5` is available by running `$ pydoc pymd5`. To follow along with these examples, run Python3 in interactive mode (`$ python3 -i`) and run the command `from pymd5 import md5, padding`.

Consider the string "Use HMAC, not hashes". We can compute its MD5 hash by running:

```
m = b"Use HMAC, not hashes"
h = md5()
h.update(m)
print(h.hexdigest())
```

or, more compactly, `print(md5(m).hexdigest())`. The output should be:

```
3ecc68efa1871751ea9b0b1a5b25004d
```

MD5 processes messages in 512-bit blocks, so internally the hash function pads m to a multiple of the 512-bit length. The padding consists of a 1 bit, followed by as many 0 bits as necessary, followed by a 64-bit count of the number of bits in the unpadded message. (If the 1 and count won't fit in the current block, an additional block is added.) You can use the function `padding(count)` in the `pymd5` module to compute the padding that will be added to a $count$ -bit message.

Even if we didn't know m , we could compute the hash of longer messages of the general form $m + \text{padding}(\text{len}(m)*8) + \text{suffix}$ by setting the initial internal state of our MD5

function to MD5(m) instead of the default initialization value, and setting the function's message length counter to the size of m plus the padding (a multiple of the block size). To find the padded message length, guess the length of m and run `count = (length_of_m + len(padding(length_of_m*8)))*8`.

The `pymd5` module lets you specify these parameters as additional arguments to the `md5` object:

```
h = md5(state="3ecc68efa1871751ea9b0b1a5b25004d", count=512)
```

Now you can use length extension to find the hash of a longer string that appends the suffix "Good advice". Simply run:

```
x = b"Good advice"
h.update(x)
print(h.hexdigest())
```

to execute the compression function over x and output the resulting hash. Verify that it equals the MD5 hash of $m + \text{padding}(\text{len}(m)*8) + x$. Notice that, due to the length-extension property of MD5, we didn't need to know the value of m to compute the hash of the longer string—all we needed to know was m 's length and its MD5 hash.

This component is intended to introduce length extension and familiarize you with the MD5 module we will be using; you will not need to submit anything for it.

Conduct a Length Extension Attack

Files

1. 3.2.1_query.txt: query
2. 3.2.1_command3.txt: command3

One example of when length extension causes a serious vulnerability is when people mistakenly try to construct something like an HMAC by using `hash(secret || message)`, where `||` indicates concatenation. **For example** (this link leads nowhere), Professor Vuln E. Rabble has created a web application with an API that allows client-side programs to perform an action on behalf of a user by loading URLs of the form:

```
https://courses.grainger.illinois.edu/ece422/fa2023/project3/api?token=
b301afea7dd96db3066e631741446ca1&user=admin&command1=ListFiles&command2=NoOp
```

where `token` is `MD5(user's 8-character password || user=.... [the rest of the URL starting from user=])`. The domain name is given as an example, we did not set up a web server for this assignment.

Text files with the query of the URL `3.2.1_query.txt` and the command line to append `3.2.1_command3.txt` are provided. Using the techniques that you learned in the previous section and without guessing the password, apply length extension to create a new query in the URL ending

with the command specified in the file, &command3=DeleteAllFiles, that is treated as valid by the server API. Submit this in the file named sol_3.2.1.txt

This is a hint about what your final solution will look like:

```
token=[updated MD5 hash]&user=admin&command1=OriginalCMD[percent-encoded padding]&command3=Reboot
```

Create a Python3 script named sol_3.2.1.py that takes as a command line argument a filename containing a valid query in the URL and modifies it such that it will execute a DeleteAllFiles command as the user, then output the new query to a specified file. You may assume that the query string will always begin with the token. You SHOULD validate that your solution works by building test cases of your own—where you know the secret key.

We will run your script as follows:

```
$ python3 your_script.py query_file command3_file output_file
```

Hint: You will want to use the quote_from_bytes() function from Python's urllib.parse module to encode non-ASCII characters in the padding.

Historical connection: In 2009, security researchers found that the API used by the photo-sharing site Flickr suffered from a length-extension vulnerability almost exactly like the one in this exercise.

What to submit Your Python3 script in sol_3.2.1.py and the modified query in sol_3.2.1.txt.

3.2.2 MD5 Collisions (15 points)

(Difficulty: Medium)

MD5 was once the most widely used cryptographic hash function, but today it is considered dangerously insecure. This is because cryptanalysts have discovered efficient algorithms for finding *collisions*—pairs of messages with the same MD5 hash value.

The first known collisions were announced on August 17, 2004 by Xiaoyun Wang, Dengguo Feng, Xuejia Lai, and Hongbo Yu. Here's one pair of colliding messages they published:

Message 1:

```
d131dd02c5e6eec4693d9a0698aff95c 2fcab58712467eab4004583eb8fb7f89
55ad340609f4b30283e488832571415a 085125e8f7cdc99fd91dbdf280373c5b
d8823e3156348f5bae6dacd436c919c6 dd53e2b487da03fd02396306d248cda0
e99f33420f577ee8ce54b67080a80d1e c69821bcb6a8839396f9652b6ff72a70
```

Message 2:

```
d131dd02c5e6eec4693d9a0698aff95c 2fcab50712467eab4004583eb8fb7f89
55ad340609f4b30283e4888325f1415a 085125e8f7cdc99fd91dbd7280373c5b
d8823e3156348f5bae6dacd436c919c6 dd53e23487da03fd02396306d248cda0
e99f33420f577ee8ce54b67080280d1e c69821bcb6a8839396f965ab6ff72a70
```

Convert each group of hex strings into a binary file.
(On Linux, run `$ xxd -r -p file.hex > file`.)

1. What are the MD5 hashes of the two binary files? Verify that they're the same.
(`$ openssl dgst -md5 file1 file2`)
2. What are their SHA-256 hashes? Verify that they're different.
(`$ openssl dgst -sha256 file1 file2`)

This component is intended to introduce you to MD5 collisions; you will not submit anything for it.

Generating Collisions Yourself

In 2004, Wang's method took more than 5 hours to find a collision on a desktop PC. Since then, researchers have introduced vastly more efficient collision finding algorithms. You can compute your own MD5 collisions using a tool written by Marc Stevens that uses a more advanced technique. You can download the `fastcoll` tool here:

http://www.win.tue.nl/hashclash/fastcoll_v1.0.0.5.exe.zip (Windows executable)
or

http://www.win.tue.nl/hashclash/fastcoll_v1.0.0.5-1_source.zip (source code)

If you are building `fastcoll` from source, you can compile using this command:

```
g++ -I /usr/local/include -L /usr/local/lib -O3 *.cpp -lboost_filesystem -lboost_program_options  
-lboost_system -o fastcoll
```

You will also need the Boost libraries. On Ubuntu, you can install these using `apt-get install libboost-all-dev`. On macOS, you can install Boost via the Homebrew package manager using `brew install boost`.

1. Generate your own collision with this tool. How long did it take?
(`$ time ./fastcoll -o file1 file2`)
2. What are your files? To get a hex dump, run `$ xxd -p file`.
3. What are their MD5 hashes? Verify that they're the same.
4. What are their SHA-256 hashes? Verify that they're different.

This component is intended to introduce you to MD5 collisions; you will not submit anything for it.

A Hash Collision Attack

The collision attack lets us generate two messages with the same MD5 hash and any chosen (identical) prefix. Due to MD5's length-extension behavior, we can append any suffix to both messages and know that the longer messages will also collide. This lets us construct files that differ only in a binary "blob" in the middle and have the same MD5 hash, i.e. $prefix \parallel blob_A \parallel suffix$ and $prefix \parallel blob_B \parallel suffix$.

We can leverage this to create two programs that have identical MD5 hashes but wildly different behaviors. We'll use Python, but almost any language would do. Copy and paste the following three lines into a file called `prefix`: (Note: typing the below lines yourself may lead to an encoding mismatch and an error may occur when running the resulting Python3 code)

```
#!/usr/bin/env python3
# -*- coding: latin-1 -*-
blob = ""
```

and put these three lines into a file called `suffix`:

```
"""
from hashlib import sha256
print(sha256(blob.encode()).hexdigest())
```

Now use `fastcoll` to generate two files with the same MD5 hash that both begin with `prefix`. (`$ ./fastcoll -p prefix -o col1 col2`). Then append the suffix to both (`$ cat col1 suffix > file1.py; cat col2 suffix > file2.py`). Verify that `file1.py` and `file2.py` have the same MD5 hash but generate different output.

Extend this technique to produce another pair of programs, `good` and `evil`, that also share the same MD5 hash. One program should execute a benign payload: `print("I come in peace.")` The second should execute a pretend malicious payload: `print("Prepare to be destroyed!")`. Note that we may rename these programs before grading them.

What to submit Two Python3 scripts named `sol_3.2.2_good.py` and `sol_3.2.2_evil.py` that have the same MD5 hash, have different SHA-256 hashes, and print the specified messages.

3.2.3 Exploiting a Padding Oracle (25 points)

(Difficulty: Hard)

In the `3.2.3_ciphertext.hex` file, you will find the hex representation of an AES-encrypted message. The first 16 bytes are the IV and the rest is ciphertext. Your goal, of course, is to find a way to decrypt this ciphertext without learning the AES key.

Before encrypting, the plaintext was padded to a multiple of 16 bytes using a custom padding scheme as follows: The first byte of padding is `0x10`, the next padding byte is `0x0f`, the next is `0x0e`, and so on, until a multiple of 16 bytes is reached. If the plaintext was already a multiple of 16 bytes, then the entire 16-byte sequence `{0x10, 0x0f, 0x0e, ..., 0x01}` is appended to the plaintext. Thus the following examples:

- "a" becomes "a\x10\x0f...\x02"
- "abcde" becomes "abcde\x10\x0f...\x06"
- "abcdefghijklmnop" becomes "abcdefghijklmnop\x10\x0f...\x01"

The following Python3 code implements this padding scheme:

```
def pad(msg):
    n = len(msg) % 16
    return msg + ''.join(chr(i) for i in range(16, n, -1))
```

The web application located at `http://72.36.89.21:8080/` can be used to check the integrity of your ciphertext. You will need to be on the campus network or VPN to access this. It reads the ciphertext in hex from the URL query string, decrypts it using the secret key, removes the padding, and confirms whether or not the resulting plaintext corresponds to the Jeopardy clue you must provide as the solution to this task. For example, running the following shell command:

```
$ curl http://72.36.89.21:8080/mp3/${netid}/?$(cat 3.2.3_ciphertext.hex)
```

should return a response with HTTP status code 200 OK and containing the string:

Correct ciphertext!

If the ciphertext is incorrect, or if the web application encounters any error while decrypting, then you will receive an error code.

We have also provided a local version for you to see how our server works. You can run it with the command:

```
$ python2 ece422-mp3-paddingoracle-server-standalone.py
```

You might think that this integrity checker isn't much help to you. But actually, the padding scheme and the error reporting of the web application interact in a devastating way. Your task is to use the technique described in Vaudenay's 2002 paper¹ to recover the plaintext.

For your reference, the following Python3 code will load a URL and print the HTTP status code:

```
import urllib.request, urllib.error

def get_status(u):
    try:
        resp. = urllib.request.urlopen(u)
        print(resp.read())
    except urllib.error.HTTPError as e:
        print(e, e.code)
```

What to submit Your Python3 script in `sol_3.2.3.py` and the decrypted message in `sol_3.2.3.txt`.

3.2.4 Mining your Ps and Qs (25 points)

(Difficulty: Hard)

The “Pretty Bad Privacy” encryption tool, `pbp.py`, can be used to insecurely encrypt files to a 1024-bit RSA public key.²

¹https://www.iacr.org/archive/eurocrypt2002/23320530/cbc02_e02d.pdf

²PBP is a “hybrid encryption” mode. It uses 1024-bit RSA (with OAEP padding, rather than textbook RSA), to encrypt a random 256-bit key, and then uses this as an AES key to encrypt the (padded) message.

Each line of the `moduli.hex` file contains a 1024-bit RSA modulus, 10,000 of these in total. You can download this from the "assignments" tab on the course website.

In `3.2.4_ciphertext.enc.asc` you have been provided the ciphertext of a Jeopardy clue, which has been encrypted using PBP with one of the RSA moduli in the file, and public exponent $e = 65537$. Factoring *any* of the 1024-bit moduli before the assignment is due is infeasible; furthermore you don't even know which one to start on!

Sometimes, badly malfunctioning implementations of RSA fail to generate unique prime numbers. The RSA moduli in the provided list were generated without sufficient entropy, and some of them share common factors. If two RSA moduli share a common factor, it is trivial to compute their GCD and factor both moduli. Unfortunately, looping over all pairs of moduli does not scale well, so you'll have some difficulty finishing the project unless you use a more efficient algorithm.

Your task is to use the method described in the "Mining your Ps and Qs" paper,³ Section 3.3, to compute the pairwise GCDs of the RSA keys provided. Once you have discovered some RSA private keys, you can then attempt to use them to recover the RSA-encrypted AES session key and decrypt the rest of homework file and submit the plaintext in `sol_3.2.4.txt`. You may use the `decrypt()` function we provided in `pbp.py`. As parameters, it expects a `Crypto.PublicKey.RSA` object and the ciphertext we gave you.

What to submit Your python3 script in `sol_3.2.4.py` and the decrypted message in `sol_3.2.4.txt`.

³<https://factorable.net/weakkeys12.extended.pdf>

Checkpoint 2: Submission Checklist

The following empty files for Checkpoint 2 have been created in your git repository under the directory Crypto. Put your solutions inside the corresponding files then commit it to git. All .hex and .txt files MUST be submitted as ASCII plaintext, and any lines in .txt and .hex submissions that begin with '#' will be ignored.

- sol_3.2.1.py
- sol_3.2.1.txt
- sol_3.2.2_good.py
- sol_3.2.2_evil.py
- sol_3.2.3.txt
- sol_3.2.3.py
- sol_3.2.4.txt
- sol_3.2.4.py

Example content of a .txt solution file

```
# this line is ignored
SPN WMKTQIW QR SPBW HQGRSEMW HQVS QY VEKW
```

Example content of a .hex solution file

```
# this line is also ignored
3dab821d92b5ca7f48beee066996b8abc82f7e5646a0561710ea5bc11c80d
```